

Memoria proyecto:

Juan de Colonia SocialClub

C.I.F.P. "JUAN DE COLONIA"

Departamento de Informática

2º DAW

Sergio Alonso López

Ana Rosa

17/05/2026

Contenido

Contenido.....	2
Índice de figuras	3
Índice de tablas	4
1. Introducción.....	5
1.1. Descripción	5
1.2. Justificación	5
2. Planificación	5
2.1. Requisitos	5
2.1.1. Requisitos funcionales	5
2.1.2. Requisitos no funcionales	5
2.2. Recursos.....	6
2.2.1. Recursos hardware	6
2.2.2. Recursos software.....	6
2.3. Planificación temporal	6
2.4. Planificación económica	7
3. Tecnologías	8
4. Desarrollo y secuenciación temporal	10
4.1. Diseño	10
4.1.1. Diagrama E/R (entidad/relación)	10
4.1.2. Modelo de datos NoSQL (Orientado a Documentos).....	10
4.1.3. Patrones de diseño del software.....	16
4.1.4. Diseño de interfaces.....	19
4.2. Pruebas.....	23
4.2.1. Pruebas funcionales	23
4.2.2. Pruebas no funcionales	24
5. Conclusiones finales	25
5.1. Grado de cumplimiento de los requisitos fijados	25
5.2. Propuestas de mejora o ampliaciones futuras	25
6. Referencias	26

Índice de figuras

Ilustración 4-1 Visualización Posts	15
Ilustración 4-2 Visualización Relaciones	15
Ilustración 4-3 Visualización Usuarios.....	16
Ilustración 4.1-4-4 Ejemplo código Modelo.....	17
Ilustración 4.1-4-5 Ejemplo código Controlador.....	18
Ilustración 4.1-4-6 Ejemplo código Vista.....	19
Ilustración 4.1.4-4-7 Boceto pantalla principal	21
Ilustración 4.1.4-4-8 Boceto pantalla Post	22
Ilustración 4.1.4-4-9 Boceto pantalla crear post	23

Índice de tablas

Tabla 2-1: Planificación temporal del proyecto.....	6
Tabla 2-2- Presupuesto del proyecto	7
Tabla 4.3 Colección de Usuarios	10
Tabla 4.4 Colección de Posts.....	12
Tabla 4.5 Subcolección de comentarios	13
Tabla 4.6 Subcolección de likes	13
Tabla 4.7 Colección de Seguidores	14

1. Introducción

1.1. Descripción

Este proyecto trata de crear una aplicación social para que nosotros los estudiantes podamos comunicarnos con otros compañeros, ya sean dentro de nuestras mismas instituciones u otras distintas. Esta aplicación permite subir posts con imágenes y textos que puedan verse sin necesidad de logearse; la creación de posts estará exclusiva para gente que tenga una cuenta creada. Utiliza tecnologías como React, TypeScript y Firebase.

1.2. Justificación

Se creo esta aplicación por la necesidad de conectar a estudiantes de distintos centros para fomentar el aprendizaje colaborativo a través del intercambio de vivencias. La plataforma abre las aulas digitalmente, facilitando que los alumnos compartan de primera mano sus proyectos, aciertos y errores.

2. Planificación

2.1. Requisitos

2.1.1. Requisitos funcionales

- ✓ Creación de cuenta
- ✓ Inicio y cierre de sesión
- ✓ Creación y publicación de Posts
- ✓ Visualización de Posts de manera publica
- ✓ Marcar me gusta en un Post
- ✓ Sistema de comentarios
- ✓ Sistema de seguidores entre usuarios
- ✓ Editar el perfil de un usuario

2.1.2. Requisitos no funcionales

- ✓ Paginas responsive para móviles y otros dispositivos
- ✓ Scroll infinito de Posts
- ✓ Permite subir imágenes de hasta 20Mb
- ✓ Bloque de usuario a zonas dependiendo de su estado

2.2. Recursos

2.2.1. Recursos hardware

Ordenador principal: Intel i7 11700KF, 32GB Ram ddr4, 3 TB de disco duro m.2, utilizado para la programación y despliegue en local del proyecto

Dispositivos de pruebas: Ordenador principal y móvil personal para probar el responsive

2.2.2. Recursos software

Lista de recursos software

- ✓ IDE: Antigravity Entorno de desarrollo desarrollado por Google [1]
- ✓ Control de versiones Git y GitHub para alojar el repositorio [2]
- ✓ Navegador web: Google Chrome utilizado para depuración y pruebas [3]

2.3. Planificación temporal

Tabla 2-1: Planificación temporal del proyecto

Descripción de la tarea	Nº de horas
Fase 1: Análisis y Definición de Requisitos: Estudio de necesidades, diseño de la arquitectura y modelado de datos en Firestore.	20
Fase 2: Configuración del Entorno: Migración a Next.js (App Router), configuración de Firebase SDK e inicialización de Tailwind CSS.	15
Fase 3: Gestión de Usuarios y Seguridad: Implementación de Login, Registro, Recuperación de contraseña y seguridad con Admin SDK.	20
Fase 4: Core de la Red Social: Desarrollo del Feed principal, lógica de publicaciones y sistema de scroll infinito.	35
Fase 5: Interacciones y Perfiles: Implementación del sistema de "Likes" persistentes en tiempo real y páginas de perfil dinámicas.	30
Fase 6: Diseño de Interfaz y UX: Modernización estética, creación de componentes premium y optimización de la responsividad móvil.	25
Fase 7: Pruebas y Despliegue: Integración continua, despliegue en Firebase Hosting y pruebas de usuario finales.	15

Fase 8: Documentación: Redacción de la memoria técnica y preparación de la presentación.	20
TOTAL HORAS	180

2.4. Planificación económica

Tabla 2-2- Presupuesto del proyecto

Descripción de la tarea	Precio (€)
Mano de obra: Desarrollo realizado por un Programador Junior (180h a 12,00 €/h).	2.160,00
Dominio: Registro de dominio personalizado (.com / .es).	15,00
Hosting: Alojamiento web con soporte para SSR (Firebase/Vercel).	120,00
Amortización: Desgaste de hardware (Laptop y periféricos).	100,00
TOTAL (€)	2395,00 €

3. Tecnologías

Para el desarrollo de esta red social se ha optado por un stack tecnológico moderno, robusto y escalable, priorizando el rendimiento y la experiencia de usuario. A continuación, se detallan las tecnologías empleadas:

Lenguajes de programación

- **TypeScript:** Es el lenguaje principal del proyecto. Actúa como un superconjunto de JavaScript que añade tipado estático. Su uso ha sido fundamental para definir las interfaces de los datos (como los tipos Post, cliente y PostComment en /src/types/index.ts) y evitar errores en tiempo de desarrollo. [4]

Frameworks y Librerías (Frontend y Backend)

- **React (v19):** Biblioteca de JavaScript de código abierto creada por Facebook, enfocada en la construcción de interfaces de usuario interactivas mediante componentes reutilizables. [5]
- **Next.js (v16):** Framework de React utilizado como núcleo del proyecto. Se ha hecho uso de su moderno enrutador App Router y de las Server Actions (acciones de servidor). Esto ha permitido unificar el frontend y la lógica de backend en un mismo repositorio, mejorando la seguridad (ocultando lógica sensible al cliente) y el SEO. [6]
- **Tailwind CSS (v4):** Framework CSS "utility-first". Se ha utilizado para crear todo el diseño responsivo y la estética personalizada ("Premium & Bold") directamente desde el código HTML/TSX, configurando variables de entorno como los colores corporativos (Midnight y Gold) en el archivo global de estilos. [7]

Base de Datos y Servicios en la Nube (BaaS)

- **Firebase:** Plataforma de desarrollo de Google que actúa como Backend as a Service (BaaS). En este proyecto se han utilizado tres de sus módulos principales: [8]
 - **Firebase Authentication:** Para el registro e inicio de sesión seguro de los usuarios. [9]
 - **Cloud Firestore:** Base de datos NoSQL orientada a documentos, utilizada para almacenar los usuarios, publicaciones, likes, comentarios y seguidores en tiempo real. [10]

- **Cloud Storage:** Utilizado para almacenar las imágenes (fotos de perfil, portadas y adjuntos de los posts) y servir las a través de URLs públicas. [11]
- **Firebase Admin SDK:** SDK de administración utilizado en el entorno de servidor (Server Actions de Next.js). Permite saltarse las reglas de seguridad de cliente para realizar operaciones críticas de forma segura, como la creación inicial de usuarios o la eliminación en cascada de datos. [12]

Librerías Auxiliares de Interfaz

- **Lucide React:** Biblioteca de iconos vectoriales (SVG) de código abierto. Se ha utilizado para toda la iconografía de la aplicación (corazones, comentarios, flechas de navegación, etc.), manteniendo una estética limpia y coherente. [13]
- **Sonner:** Biblioteca de notificaciones (toast notifications) altamente personalizable. Se utiliza para dar feedback inmediato al usuario ante cualquier acción, como confirmar que un post se ha subido correctamente o alertar de un error en el inicio de sesión. [14]

4. Desarrollo y secuenciación temporal

4.1. Diseño

4.1.1. Diagrama E/R (entidad/relación)

A diferencia de los sistemas relacionales tradicionales (SQL) basados en tablas rígidas y restricciones de integridad mediante claves foráneas, la persistencia de *Juan de Colonia SocialClub* se ha construido sobre **Cloud Firestore**. Firestore es una base de datos NoSQL orientada a documentos que encaja de forma nativa con el ecosistema de aplicaciones web modernas y reactivas

4.1.2. Modelo de datos NoSQL (Orientado a Documentos)

A diferencia de las bases de datos SQL tradicionales, este proyecto utiliza Cloud Firestore (Firebase), una base de datos NoSQL orientada a documentos. En este paradigma, los datos se estructuran en Colecciones que contienen Documentos (similares a objetos JSON), los cuales a su vez pueden contener Subcolecciones.

El diseño se ha pensado para optimizar las lecturas (reads) y permitir la escalabilidad en tiempo real. A continuación, se detalla la estructura principal de la base de datos:

1. Colección: usuarios

Almacena el perfil completo de cada estudiante registrado. El ID del documento se mapea directamente con el UID de Firebase Authentication.

Tabla 4.3 Colección de Usuarios

Campo	Tipo de Datos	Descripción / Restricciones
uid	string	Identificador único del usuario (Clave Primaria / Document ID).

Campo	Tipo de Datos	Descripción / Restricciones
nombre	string	Nombre de pila del estudiante.
apellidos	string	Apellidos del estudiante.
usuario	string	Nickname o nombre de usuario en la plataforma (Único).
email	string	Correo institucional (Restringido a @educa.jcyl.es).
biografía	string	Breve descripción o estado del perfil del alumno.
fotoPerfil	string	URL pública de la imagen alojada en Cloud Storage.
fotoPortada	string	URL pública de la imagen de cabecera alojada en Cloud Storage.
creación	timestamp	Fecha y hora de registro de la cuenta.
seguidores	number (int)	Contador de usuarios que siguen a este perfil (Actualizado mediante transacciones atómicas).
seguidos	number (int)	Contador de perfiles a los que sigue el usuario (Actualizado mediante transacciones atómicas).
emailVerificado	boolean	Flag de control que indica si se validó el código de 6 dígitos.

2. Colección posts:

Almacena todas las publicaciones globales de la red social.

Tabla 4.4 Colección de Posts

Campo	Tipo de Datos	Descripción / Restricciones
id_post	string	Identificador autogenerado por Firestore (Document ID).
contenido	string	Bloque de texto redactado por el alumno.
adjunto	string	URL opcional de la imagen asociada guardada en Cloud Storage.
likes	number (int)	Contador numérico persistente de los "Me gusta" recibidos.
numComments	number (int)	Contador de comentarios totales del post.
createdAt	timestamp	Fecha y hora de creación (Empleado para la ordenación secuencial del Feed).
creador	map (object)	Datos desnormalizados del autor del post (Optimiza lecturas sin necesidad de joins complejos).
uid	string	UID único del autor.
usuario	string	Nickname del autor.
nombre	string	Nombre del autor.
fotoPerfil	string	URL de su foto actual.

2.1 Subcolección: posts/{idPost}/comments

Subcolección anidada dentro de cada post que organiza las respuestas y debates de los estudiantes.

Tabla 4.5 Subcolección de comentarios

Campo	Tipo de Datos	Descripción / Restricciones
id_comentario	string	Identificador único del comentario (Document ID).
contenido	string	Mensaje de texto introducido por el comentarista.
createdAt	timestamp	Sello de tiempo de la publicación del comentario.
creador	map (object)	Datos embebidos desnormalizados del creador del comentario.

2.2. Subcolección: posts/{idPost}/likes

Colección interna que gestiona qué alumnos interactuaron con el post.

Tabla 4.6 Subcolección de likes

Campo	Tipo de Datos	Descripción / Restricciones
uid	string	El ID del documento coincide con el UID del usuario que dio el me gusta (evita duplicados de forma nativa).
createdAt	timestamp	Registro temporal del momento del "like".

3. Colección seguidores:

Almacena de forma independiente las relaciones de seguimiento recíproco (Muchos a Muchos) de la red social.

Tabla 4.7 Colección de Seguidores

Campo	Tipo de Datos	Descripción / Restricciones
id_relacion	string	Compuesto por seguidorId_seguidoId para garantizar restricciones únicas (Document ID).
seguidorId	string	UID de la persona que decide seguir (Referencia a la colección usuarios).
seguidoId	string	UID del perfil que recibe el seguidor (Referencia a la colección usuarios).
creadoEn	timestamp	Sello de tiempo de inicio de la relación de seguimiento.

4. Colecciones de Sistema (mail y verificationCodes):

a. Colección: verificationCodes

Depósito volátil para la validación multifactor durante el registro.

- i. **id_doc (string)**: Código alfanumérico o autogenerado.
- ii. **code (string / number)**: Código numérico de verificación de 6 dígitos enviado por email.
- iii. **email (string)**: Dirección a verificar.

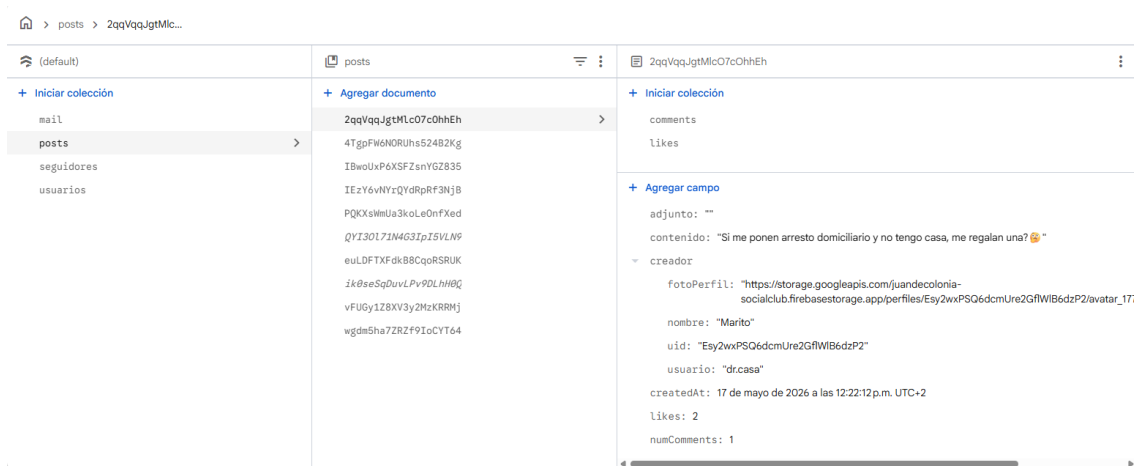
b. Colección: mail

Cola de mensajería asíncrona procesada por la extensión de Firebase para despachar plantillas HTML de correo.

- i. **to (string)**: Dirección del destinatario de la institución educativa.

- ii. message (map): Contiene el subject (asunto) y el html estructurado de bienvenida o código de seguridad.

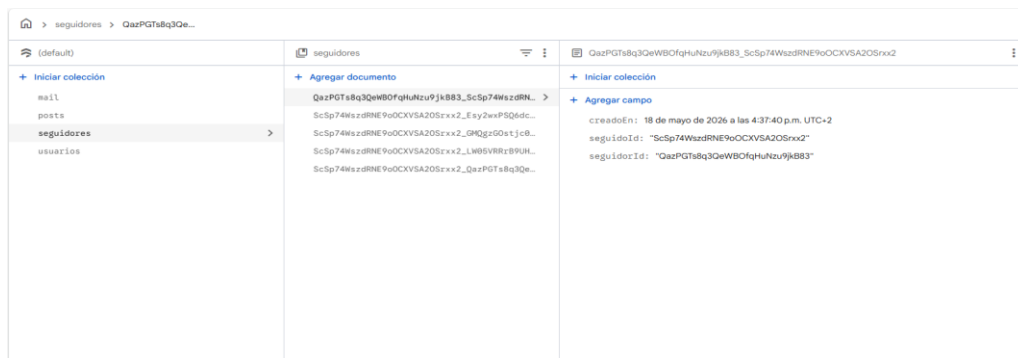
Ejemplos de la base de datos:



posts	2qqVqqJgtMlc07c0hhEh
4TgpFW6NORUhs524B2Kq IBwoUxP6XSFZsnYGZ835 IEzY6vNYrQYdRpRf3NjB PQKXsWmUa3koLeOnfXed QYI30L71M63IpI5VLN9 euLDFTXfDk88CqRSRUk ik8seSqDuvLpV9DLhH9Q vFUGy1Z8XV3y2HzKRRmj wgdeSha7ZRZf9IcCYT64	adjunto: "" contenido: "Si me ponen arresto domiciliario y no tengo casa, me regalan una? 🏠" creador fotoPerfil: "https://storage.googleapis.com/juandecolonia-socialclub.firebaseio.com/profiles/Esy2wxPSQ6dcmUre2GRWIB6dzP2/avatar_177" nombre: "Marito" uid: "Esy2wxPSQ6dcmUre2GRWIB6dzP2" usuario: "dr.casa" createdAt: 17 de mayo de 2026 a las 12:22:12 p.m. UTC+2 likes: 2 numComments: 1

Ilustración 4-1 Visualización Posts

(Visualización de un post y sus subcolecciones de likes y comentarios)



seguidores	QazPGTs8q3QeWBOfgHuNzu9jkB83_ScSp74WszdRNE9oOCKVSA2OSrx2
QazPGTs8q3QeWBOfgHuNzu9jkB83_ScSp74WszdRNE9oOCKVSA2OSrx2 ScSp74WszdRNE9oOCKVSA2OSrx2_Esy2wxPSQ6dc... ScSp74WszdRNE9oOCKVSA2OSrx2_GM0gG0stjC0... ScSp74WszdRNE9oOCKVSA2OSrx2_LW8VVRr8PUH... ScSp74WszdRNE9oOCKVSA2OSrx2_QazPGTs8q3Qe...	creadoEn: 18 de mayo de 2026 a las 4:37:40 p.m. UTC+2 seguidoId: "ScSp74WszdRNE9oOCKVSA2OSrx2" seguidorId: "QazPGTs8q3QeWBOfgHuNzu9jkB83"

Ilustración 4-2 Visualización Relaciones

(Visualización de las relaciones que se generar al seguir a un usuario)

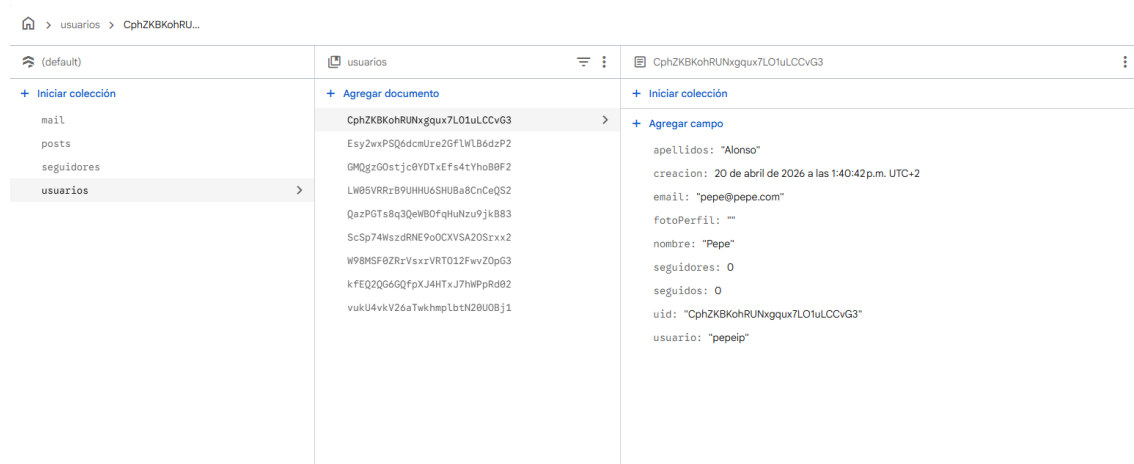


Ilustración 4-3 Visualización Usuarios

(Visualización del documento de un usuario verificado)

4.1.3. Patrones de diseño del software

Justificación de patrones utilizados.

1. Arquitectura MVC (Modelo-Vista-Controlador) adaptada:

- Modelo: Definido por las interfaces de TypeScript en src/types y la estructura de documentos en Firebase Firestore. Representa la lógica de datos.
- Vista: Implementada con componentes de React (Next.js) y estilizada con Tailwind CSS. Se encarga de la interfaz de usuario.
- Controlador: Gestionado a través de las "Server Actions" de Next.js. Estas acciones contienen la lógica para procesar formularios, interactuar con la base de datos y manejar la autenticación

2. Patrón Singleton: Utilizado en la inicialización de Firebase. Se asegura que la instancia de la aplicación Firebase y el acceso a Firestore se inicien una única vez, optimizando el uso de recursos tanto en el cliente como en el servidor.

3. Patrón Observer: Implementado de forma nativa a través de los "listeners" de Firestore (onSnapshot). Esto permite que la interfaz de usuario se actualice automáticamente cuando hay cambios en la base de datos (por ejemplo, al recibir un nuevo Like o comentario en tiempo real).

4. Custom Hooks (Hooks Personalizados): Se ha extraído la lógica compleja del estado en hooks específicos (directorio src/hooks), facilitando la reutilización de código y manteniendo los componentes de la vista limpios.

Organización del código y estructura de la aplicación

La aplicación utiliza la estructura de carpetas de App Router de Next.js, lo que permite una organización modular y escalable:

- a. /src/app: Contiene las rutas (páginas) y las "Server Actions".
- b. /src/components: Componentes reutilizables de la interfaz.
- c. /src/firebase: Configuración y servicios de inicialización de Firebase.
- d. /src/hooks: Lógica de negocio extraída del ciclo de vida de React.
- e. /src/types: Definiciones de tipos para asegurar la integridad de los datos en todo el proyecto.

Ejemplo de código

Modelo (Definición de datos en types/index.ts):

```
export type Post = {  
  id:string;  
  creador: {  
    uid: string;  
    nombre: string;  
    usuario: string;  
    fotoPerfil: string;  
  }  
  contenido: string;  
  adjunto: string;  
  likes: number;  
  numComments?: number;  
  createdAt: Timestamp;  
  likedByMe?: boolean;  
};
```

Ilustración 4.1-4-4 Ejemplo código Modelo

Controlador (serverAction en app/actions/subirPost.ts):

```

try {
  /**
   * PASE 1: Obtener la identidad del "creador"
   * No confiamos solo en lo que mande el cliente. Buscamos el perfil real
   * en nuestra base de datos para asegurar que el nombre y la foto son correctos.
   */
  console.log(`[subirPost] 🔍 Consultando en Firestore el documento del usuario: "usuarios/${uid}"`);
  const userDoc = await adminFirestore.collection("usuarios").doc(uid).get();
  if (!userDoc.exists) {
    console.warn(`[subirPost] ⚠️ No se encontró el usuario en la base de datos para el UID: "${uid}"`);
    throw new Error("Lo sentimos, no hemos podido encontrar tu perfil de usuario.");
  }
  const userData = userDoc.data();
  console.log(`[subirPost] ✅ Usuario encontrado. Nombre: "${userData?.nombre}", Usuario: "${userData?.usuario}"`);

  // Preparamos el objeto del creador con datos por defecto por si falta alguno
  const creador = {
    fotoPerfil: userData?.fotoPerfil || "",
    nombre: userData?.nombre || "Usuario",
    usuario: userData?.usuario || "anónimo",
    uid: uid
  };
}

/**
 * PASE 2: Gestión de archivos adjuntos
 * Si el usuario ha seleccionado una imagen, la procesamos y la subimos a Storage.
 */
if (imagen && imagen.size > 0) {
  console.log(`[subirPost] 📁 Procesando archivo adjunto para subir a Cloud Storage...`);
  // Convertimos el archivo a un Buffer que Node.js pueda manejar
  const arrayBuffer = await imagen.arrayBuffer();
  const buffer = Buffer.from(arrayBuffer);

  // Creamos una ruta organizada: posts / ID_USUARIO / timestamp_nombre.ext
  const safeFileName = imagen.name.replace(/[^a-z0-9.]/gi, "_").toLowerCase();
  const filePath = `posts/${uid}/${Date.now()}_${safeFileName}`;
  console.log(`[subirPost] 📁 Destino en Storage: "${filePath}"`);
  const fileRef = bucket.file(filePath);

```

Ilustración 4.1-4-5 Ejemplo código Controlador

Vista (Componente React):

```

<form onSubmit={handleLogin} className="space-y-6">
  <div className="space-y-2">
    <label
      htmlFor="email"
      className="input-label"
    >
      Correo Electrónico
    </label>
    <input
      id="email"
      type="email"
      value={email}
      onChange={(e) => setEmail(e.target.value)}
      required
      placeholder="tu@email.com"
    />
  </div>

  <div className="space-y-2">
    <label
      htmlFor="password"
      className="input-label"
    >
      Contraseña
    </label>
    <input
      id="password"
      type="password"
      value={password}
      onChange={(e) => setPassword(e.target.value)}
      required
    />
  </div>

  <div className="flex flex-col gap-4">
    <div className="text-right">
      <Link
        href="/createAccount"
        className="text-gold-accent hover:text-midnight transition-colors"
      >
        ¿No tienes una cuenta?
      </Link>
    </div>
  </div>

```

Ilustración 4.1-4-6 Ejemplo código Vista

4.1.4. Diseño de interfaces

Mapa del sitio (Sitemap)

La navegación de la aplicación se distribuye de la siguiente manera:

- **/ (Home):** Feed principal con publicaciones de todos los usuarios.
- **/login:** Acceso al sistema.
- **/createAccount:** Registro de nuevos socios de la aplicación.
- **/profile:** Vista del perfil personal del usuario logueado o de otros usuarios.
- **/create-post:** Editor para crear nuevas publicaciones con fotos.
- **/post/[id]:** Vista detallada de una publicación y sus comentarios.
- **/reset-password:** Recuperación de contraseña.

Guía de estilos

El diseño busca una estética "Premium & Bold", utilizando una paleta de colores sofisticada y moderna:

- **Paleta de colores:**
 - Midnight (#0f172a): Color principal para fondos oscuros y textos destacados.
 - Gold (#d97706): Color de acento para botones principales, iconos e indicadores de importancia.
 - Slate Gray: Utilizado para fondos secundarios y bordes suaves, creando profundidad.
- **Tipografía:** Se utilizan fuentes de sistema modernas (Sans-serif) con variaciones de peso (font-black para títulos y font-medium para lectura), priorizando la legibilidad y el impacto visual.
- **Componentes Visuales:**
 - card-premium: Contenedores con bordes redondeados pronunciados y sombras suaves.
 - glass-midnight: Efecto de cristal con desenfoque para menús flotantes.

Bocetos de las principales pantallas:

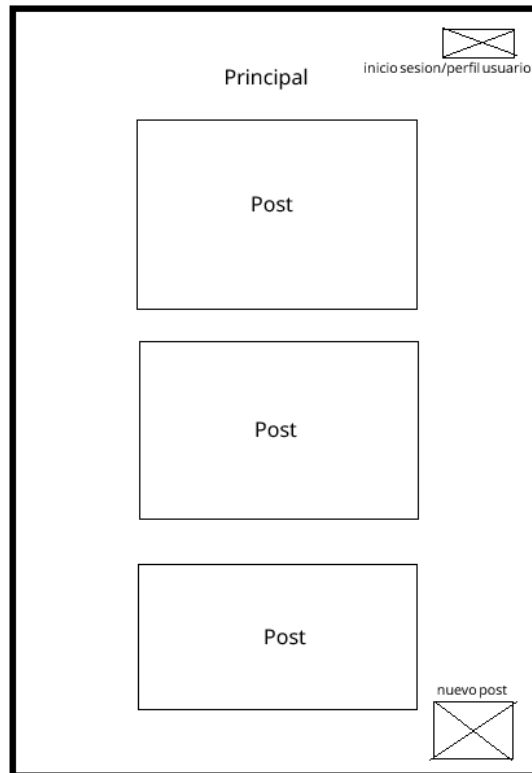


Ilustración 4.1.4-4-7 Boceto pantalla principal

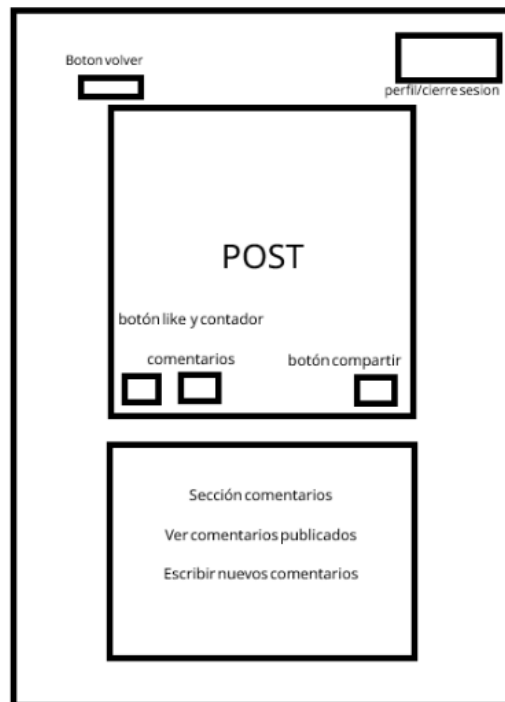


Ilustración 4.1.4-4-8 Boceto pantalla Post

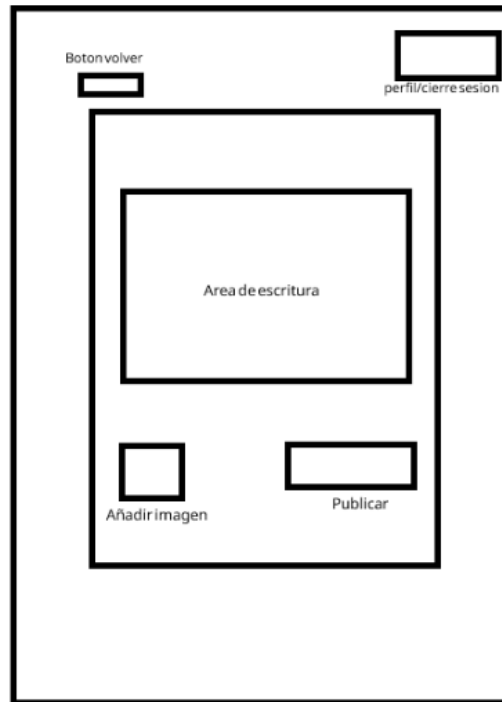


Ilustración 4.1.4-4-9 Boceto pantalla crear post

4.2. Pruebas

Para garantizar la calidad, seguridad y correcto funcionamiento de la plataforma "SocialClub", se ha llevado a cabo una batería de pruebas dividida en dos categorías principales:

4.2.1. Pruebas funcionales

Se han validado los casos de uso principales de la aplicación para asegurar que cumplen con los requisitos iniciales:

- **Autenticación y Registro:** Se ha verificado que el sistema rechaza cualquier intento de registro con un correo que no pertenezca al dominio @educa.jcyl.es. Además, se ha probado con éxito el flujo de verificación de cuentas mediante el envío y validación del código de 6 dígitos por email.
- **Gestión de Publicaciones (Posts):** Se comprobó la correcta subida de archivos de imagen a Firebase Storage y la creación del documento en Firestore. Asimismo, se testeó la eliminación en cascada: al borrar un

post, el sistema elimina correctamente su imagen asociada en Storage y todos sus comentarios anidados.

- **Interacciones en Tiempo Real:** Se probó el correcto funcionamiento de los botones de "Me gusta" y la publicación de comentarios, verificando que la interfaz se actualiza instantáneamente gracias a los listeners (onSnapshot).
- **Sistema de Seguidores:** Se sometió a prueba la lógica transaccional atómica al seguir o dejar de seguir a un usuario, comprobando que los contadores numéricos de "seguidores" y "seguidos" se actualizan sin generar inconsistencias de datos.

4.2.2. Pruebas no funcionales

Estas pruebas se enfocaron en el rendimiento, la seguridad y la experiencia de usuario:

- **Seguridad y Reglas de Acceso:** Se intentó realizar operaciones maliciosas (como borrar un post de otro usuario) desde el lado del cliente, confirmando que las reglas de seguridad de Firestore (firestore.rules) bloquean cualquier escritura o borrado no autorizado.
- **Rendimiento y Paginación (Scroll Infinito):** Se verificó el correcto funcionamiento del scroll infinito en el feed principal. Utilizando la API IntersectionObserver, se comprobó que el sistema carga las publicaciones en lotes de 10 en 10 (limit(10)), evitando sobrecargar la memoria del navegador y minimizando las lecturas a la base de datos.
- **Diseño Responsivo:** Se testeó la interfaz en diferentes resoluciones (móvil, tablet y escritorio), verificando que elementos como el menú de navegación lateral pasan correctamente a ser un menú desplegable superior en pantallas pequeñas.

5. Conclusiones finales

5.1. Grado de cumplimiento de los requisitos fijados

- ✓ El proyecto cumple exitosamente con los requisitos funcionales principales establecidos en la planificación.
- ✓ Se ha logrado implementar una plataforma funcional exclusiva para estudiantes donde pueden registrarse, interactuar mediante posts, likes y comentarios, y seguir a otros compañeros.
- ✓ Como análisis de las desviaciones, los requisitos no funcionales referentes al sistema de repostear publicaciones de otros usuarios y al sistema de mensajes privados entre usuarios no se han implementado en esta primera versión.

5.2. Propuestas de mejora o ampliaciones futuras

- ✓ Desarrollar el sistema de mensajes privados entre usuarios para permitir la comunicación directa.
- ✓ Implementar el sistema de repostear publicaciones de otros usuarios para aumentar la interacción del contenido.
- ✓ Añadir un sistema de notificaciones push en tiempo real para avisar a los usuarios cuando reciben nuevos seguidores, "me gusta" o comentarios.

6. Referencias

- [1] «Antigravity,» [En línea]. Available: <https://antigravity.google/>. [Último acceso: 2026 05 17].
- [2] «GitHub,» [En línea]. Available: <https://github.com/>. [Último acceso: 2026 05 17].
- [3] «Google Chrome,» [En línea]. Available: <https://www.google.com/intl/es-es/chrome/>. [Último acceso: 2026 05 17].
- [4] «Typescript,» [En línea]. Available: <https://www.typescriptlang.org/>. [Último acceso: 2026 05 20].
- [5] «React,» [En línea]. Available: <https://es.react.dev/>. [Último acceso: 19 04 2026].
- [6] [En línea]. Available: <https://es.wikipedia.org/wiki/Next.js>. [Último acceso: 19 04 2026].
- [7] [En línea]. Available: <https://learn.microsoft.com/es-es/shows/web-wednesday/what-is-tailwind-css>. [Último acceso: 19 04 2026].
- [8] [En línea]. Available: <https://firebase.google.com/?hl=es>. [Último acceso: 19 04 2026].
- [9] «Firebase auth,» [En línea]. Available: <https://firebase.google.com/docs/auth?hl=es-419>. [Último acceso: 20 05 2026].
- [10] «Firebase Firestore,» [En línea]. Available: <https://firebase.google.com/docs/firestore?hl=es-419>. [Último acceso: 20 05 2026].
- [11] «Storage firebas,» [En línea]. Available: <https://firebase.google.com/s/results?hl=es-419&q=storage>. [Último acceso: 20 05 2026].
- [12] «FirebaseSDK Admin,» [En línea]. Available: <https://firebase.google.com/docs/admin/setup?hl=es-419>. [Último acceso: 2026 05 20].
- [13] «Lucide,» [En línea]. Available: <https://lucide.dev/>. [Último acceso: 20 05 2026].
- [14] «Sonner,» [En línea]. Available: <https://sonner.emilkowal.ski/>. [Último acceso: 20 05 2026].

[15] [En línea]. Available: <https://es.vite.dev/guide/>. [Último acceso: 19 04 2026].